

VoiceBridge — Comprehensive Technical Overview

Author: Zahidul Islam

Repository: <https://github.com/ZahidulIslam2222/voicebridge>

Updated: 2026-10-08

Website: <https://voicebridge.zahidul-islam.com/>

This overview covers the maintained frontend/backend, operating boundaries and future engineering roadmap.

Current delivery, setup and verification

Capabilities

- Responsive public website with eight destinations and twelve operator destinations.
- Business console with Google OIDC, tenant/role checks, secure server sessions and CSRF protection.
- Availability, booking, rescheduling and cancellation with explicit confirmation, timezone validation, conflict prevention, idempotency and revision checks.
- Independent leased worker for calendar, CRM and follow-ups, with persisted retries, receipts, dead letters and explicit unknown-delivery states.
- Grounded knowledge with source revisions, bounded PDF parsing and abstention.
- Provider-authenticated Retell/Vapi tools, active-call scope and scoped MCP interfaces.

Implementation and roadmap

The authenticated frontend and backend are deployed. Recorded gates include 150 backend/tooling tests, 78 frontend tests, types/lint/security/builds, twelve backend and six frontend release-file parity checks, and twenty-one public browser routes. Persistent booking/recovery checks use controlled test data and captured follow-ups. Real voice/audio acceptance, custom-voice approval and external calendar/CRM/email receipts are the next integration gates. See current evidence.

The future architecture targets 1M+ concurrent users, separately defining sessions, API traffic, calls and mixed workloads. Reliability planning defines a 99% baseline and 99.9% production service objectives with measurable SLOs, error budgets, redundancy, staged capacity tests and recovery gates.

Start developing

Prerequisites: Node 24.15+ within Node 24, Python 3.12–3.14 and PostgreSQL. From the repository root:

```
npm ci
npm run dev
python -m venv .venv
# Activate the environment using your platform's standard command.
python -m pip install -r requirements.lock
python -m pip install --no-deps -e .
```

Populate an ignored `.env.local` from `.env.example`, configure a local PostgreSQL instance and unique local credentials, then run:

```
python -m voicebridge.cli migrate --env-file .env.local
python -m voicebridge.cli seed --env-file .env.local
python -m voicebridge.cli api --env-file .env.local
```

Run `python -m voicebridge.cli worker --env-file .env.local` in a separate process. Configure frontend API URL/origins from `.env.example` and open Business engine. Keep real transports disabled until their integration acceptance. Backend tests have their own environment requirements described in testing.

Deployment and release discipline

Release tools populate frontend/engine templates from private deployment settings. Never write actual credentials into committed templates. The backend Docker context allowlists runtime/data/migration inputs and excludes private records.

Tools: `tools/prepare-frontend-release.mjs` and `tools/prepare-engine-release.mjs`. Generated releases include immutable artifacts/manifests. Required order: inventory/ drift -> local snapshot/change/gates -> migration/rollback review -> deploy -> real acceptance -> local/live hashes. Production is a deployment target, not an editor. Preserve unrelated containers/gateway/database state; never delete working volumes.

See frontend deployment, engine operations, reliability and runbooks.

Latest publication checks

150 backend/tooling tests and 78 frontend tests pass locally. Type, lint, build, packaging and security gates pass in their declared scopes. A clean-runner correction adds SMTP capture for actual worker crash recovery. All five checks passed on GitHub, and the reviewed correction is merged.

Architecture and business boundaries

- React console → HTTPS gateway
- HTTPS gateway → FastAPI authorization and contracts
- OIDC issuer → FastAPI authorization and contracts
- Retell → FastAPI authorization and contracts
- Vapi → FastAPI authorization and contracts
- Scoped MCP clients → FastAPI authorization and contracts
- FastAPI authorization and contracts → Booking and grounded knowledge
- Booking and grounded knowledge → PostgreSQL records and jobs
- PostgreSQL records and jobs → Independent leased worker
- Independent leased worker → Calendar connector
- Independent leased worker → CRM connector
- Independent leased worker → Follow-up transport
- Passive observer → HTTPS gateway

The current controlled deployment captures calendar/CRM/follow-up outcomes; production external delivery has its own acceptance gate. Browser and native-call tools use the same authorization and business rules.

Decisions

- Decision: PostgreSQL owns bookings, idempotency and jobs; Reason: Transactions couple state and durable work; Growth constraint: Connections, locks and hot records need measured limits
- Decision: Separate leased worker; Reason: Request timeouts do not determine delivery lifetime; Growth constraint: Backlog, provider quotas and unknown outcomes need supervision
- Decision: Tenant locks and revisions; Reason: Concurrent intent cannot silently overwrite; Growth constraint: Busy tenants can serialize; measure before narrowing locks
- Decision: Opaque server sessions; Reason: Browser receives no provider key or trusted tenant assignment; Growth constraint: Session cleanup and regional routing need explicit design
- Decision: Shared Retell/Vapi services; Reason: Provider choice does not fork business policy; Growth constraint: Native schemas/audio need provider-specific verification
- Decision: Typed settings and validated data; Reason: One owner per changeable value; Growth constraint: Version configuration/content with releases
- Decision: Immutable releases and hash parity; Reason: Auditable deployment/rollback; Growth constraint: Schema compatibility must permit rollback

Flows

Booking: authenticated identity -> tenant/role -> confirmation -> idempotency/revision -> timezone/service/overlap validation -> durable reservation/calendar job -> verified calendar receipt -> CRM/follow-up jobs. Pending acceptance is distinct from external confirmation.

Knowledge: validated import -> revisioned storage -> tenant search -> source identities -> answer or abstention. Retrieved instructions cannot authorize tools.

Native call: trusted owner prepares bounded intent/capability before native creation -> returned ID bound to maintained provider/agent -> audio join -> platform-authenticated tools/events -> terminal event retires scope. Native creation/binding/audio still needs full live acceptance.

Data and tenancy

Records include contacts, bookings, calls/transcripts, documents, agent revisions, idempotency, jobs/receipts and authentication state. Services filter by tenant, including read/management paths. Caller-provided identity cannot promote scope. Database row policies, paged operator access, tenant placement and regional partitions are future scale/security gates. See API, security and scaling.

HTTP, provider and MCP contracts

core-openapi.json is the generated contract. Frontend generated types are checked against it by `npm run test:core-contracts`. Update server schemas first, regenerate using the configured CLI/type generator, then review both sides.

- Group: Health; Purpose: Liveness/schema readiness; Boundary: Minimal public status, no business data
- Group: Authentication; Purpose: OIDC/session/logout; Boundary: State/PKCE/nonce/signature and trusted subject binding
- Group: State/bookings; Purpose: Availability and confirmed operations; Boundary: Tenant role/capability; CSRF on session mutations
- Group: Knowledge; Purpose: Import/search/revise/delete; Boundary: Tenant, size/parser limits and revisions
- Group: Jobs; Purpose: Inspect/replay/reconcile; Boundary: Management scope; unknown delivery cannot be blindly retried
- Group: Evaluations; Purpose: Maintained business cases; Boundary: Authorized isolated records and cancellation cleanup
- Group: Native calls; Purpose: Intent, bind, lifecycle/tools; Boundary: Owner plus bounded capability; platform authentication first
- Group: MCP; Purpose: Business/management read/write tools; Boundary: Separate scopes and protocol-valid errors

Exact methods/routes/schemas/errors belong to OpenAPI and the provider specification. Health success does not establish user authorization. Never expose management or provider keys in browser variables.

Use stable operation IDs on retries, new IDs when intent changes, expected revisions and timezone-aware appointments. Present pending/conflict/unknown outcomes explicitly. Bound bodies, timeouts and response lists. Do not retry ambiguous provider writes or retired calls. Retell verifies raw-body signatures; Vapi verifies server credentials. Opt-in metadata capability handling follows platform verification. See native interoperability.

Authenticated core pilot verification

Updated: 2026-10-07 UTC. This is a scrubbed evidence summary derived from the private project dossier.

The current live authenticated pilot is backend03/frontend06. All twelve backend and six frontend files match local/live; fourteen runtime modules match the frozen image. Two preflight format mismatches stopped safely before live process changes; actual platform output was then verified and the reviewed reconciliation succeeded. Backend promotion preserved twenty-seven unrelated/database/capture containers; frontend promotion preserved ten unrelated gateway files.

Verified behavior

- Actual Google owner sign-in, trusted identity binding, administrator two-step verification, secure host-only session cookie and CSRF rejection passed.
- Two test-data bookings persist across page reload and worker restart. Six calendar/CRM/follow-up jobs completed once, with two captured follow-ups. External calendar/CRM delivery and external email remain disabled.
- A server database backup restored into an isolated clone with thirteen table count/hash matches; the working database was preserved.
- Prior authenticated native live Chrome passed twenty-one routes, three widths and four flows with zero JavaScript/CSP errors. Actual logout revoked the session; a copied old cookie returned 401.
- Dedicated Retell and Vapi configurations were created and read back successfully, with three business tools and sixty-second native duration limits. No native calls or audio samples have run.

Recorded release gates

- Gate: Backend; Result: 144 tests passed; 23 focused lifecycle/intent checks passed
- Gate: Frontend; Result: 78 tests passed
- Gate: Types/lint; Result: Mypy14 modules, Ruff41 files, TypeScript and ESLint passed
- Gate: Build/package; Result: Python wheel/sdist, frontend build and 11 packaging/contract checks passed
- Gate: Provider code review; Result: Fresh intent review02 ready
- Gate: Native preparation; Result: Five safety checks passed; fresh review04 ready
- Gate: Frontend promotion safety; Result: Twelve inventory/recovery checks passed; fresh review08 ready

- Gate: Exact frontend06 artifact; Result: 21 maintained-heading page assertions, login render, HTML/nginx hash binding; zero JavaScript/CSP/external requests
- Gate: Immutable image; Result: Fourteen source modules match local and frozen backend03 manifest
- Gate: Secret audit; Result: 184 public files checked against34 saved private values; no matches; private paths ignored/unstaged and environment example coverage complete
- Gate: Security; Result: Bandit0; scoped Semgrep79 applicable Python rules/14 runtime files,0findings/errors; latest edit-hook scans clean
- Gate: Dependency advisories; Result: 87 Python and243 npm dependencies,0known advisories at the recorded check

The frontend build retains its bundle-size warning. Manual shared-hook invocation is verified; automatic dispatch on every write remains unproven. Static reviews establish code findings, while runtime results are recorded separately.

Limits and remaining acceptance

Real voice samples, external business-provider delivery and real-call acceptance remain open. Seven current public HTTP checks pass: readiness200, unauthorized state/intent401, invalid Retell/Vapi authentication401 and valid-authentication/unregistered-call403. Current actual public browser also passes21route-specific headings with exact HTML hash and zero JavaScript/CSP/external requests. Read-only database/health inspection confirms one identity binding, six completed jobs, no pending jobs/sessions, two captures and101health samples/100successful/one unsuccessful with0unknown seconds at this check. A registered native assistant and passing envelopes do not prove a real conversation.

A guarded local load run measured40requests across4clients with0errors and approximately24ms p95; a separate populated-data run measured approximately188ms p95. Neither proves a million users or simultaneous calls. Passive health samples report their observed periods and unknown gaps; sustained uptime, zero downtime, off-device backup and physical power-loss durability remain unproven. The broader80-criterion production roadmap remains open.

Final aggregate source review: READY FOR VERIFIED FREE PILOT SCOPE WITH LIVEVOICE OUTSTANDING. The review independently ran no tests/deployment. A separate actual HTTPS check confirms served HTML bytes and the exact live CSP header match the local frontend06 package. The whole project and real voice are not accepted as complete.

Legal applicability, privacy and responsible voice use

This is an applicability and implementation register. Before each customer/region release, assign an owner to verify entities, purposes, recipient locations, provider contracts and legal decisions. Review date: 2026-10-08 .

- Area: EU/EEA privacy; Applicability question: Which controller/processor and people/purposes are in scope?; Release evidence: Lawful basis, notice, minimization, rights, contracts, transfers and retention
- Area: EU AI rules; Applicability question: Intended use/risk class? Are people informed of AI interaction?; Release evidence: Classification, applicable transparency, prohibited-use screen and dated review
- Area: US calls/marketing; Applicability question: Inbound service, outbound information or solicitation?; Release evidence: Federal/state consent, suppression/opt-out, calling-time and identification review
- Area: US recording; Applicability question: Locations of participants and recorder?; Release evidence: Jurisdiction consent/disclosure decision and proof/withdrawal procedure
- Area: California privacy; Applicability question: Business/statutory scope and covered processing?; Release evidence: Rights/notice assessment, contracts and applicable sale/sharing controls
- Area: Regulated sectors; Applicability question: Covered health information or regulated advice?; Release evidence: Sector review, agreements and controls before accepting that data
- Area: Voice/media rights; Applicability question: Who owns/authorized voices, recordings and source material?; Release evidence: Consent/licence scope, permitted use, duration, revocation and provider restrictions

The EU register follows GDPR and EDPB guidance. AI classification/transparency review follows the European Commission overview. Inbound service and outbound marketing need different applicability decisions; the FTC guide identifies FCC/TCPA and state-law overlap. California assessment uses Attorney General guidance. Health deployments need an independent HIPAA applicability review.

Data lifecycle

Inventory contacts, appointments, transcripts, recordings, knowledge, authentication and operational events. Assign purpose, legal basis where required, roles, provider destinations, retention and deletion owner. Avoid payment-card details, unnecessary sensitive data and free-text secrets. Public evidence excludes client identities.

Before recordings, implement disclosure, consent/refusal, private retrieval and enforced deletion of local/provider copies. Voice may be personal data; biometric treatment depends on identification purpose and applicable law.

Provide access/correction/deletion/export/restriction/objection as applicable. Verify requesters proportionately. Deletion covers replicas, indexes, queued work and providers, with lawful holds and expiring backups documented. Do not promise immediate erasure from immutable backups without the restore/deletion policy.

Ownership and release gates

Customers approve business data, recipients, consent and communication purposes. Maintainers own technical enforcement and gap records. Review provider terms, subprocessors, locations, retention, incidents and voice-submission rights. Emergency, diagnostic, credit, employment and other high-impact uses require their own reviewed boundary. Booking automation does not authorize those decisions.

Publish customer-specific notices/agreements after confirming real data flows. Include AI disclosure, human handoff, complaints, opt-out and incident procedures. Track decisions in the roadmap; this register is not a blanket certification for every jurisdiction or customer.

Security model

Controls use OWASP ASVS as a verification framework. Assess each against intended scope; scanners do not replace business-flow tests.

- Threat: Account/session theft; Current boundary: OIDC validation, opaque encrypted server sessions, secure cookies and revocation; Additional acceptance: Customer MFA/recovery and abuse tests
- Threat: Cross-tenant access; Current boundary: Server identity binding and tenant-filtered services; Additional acceptance: Expanded role matrix, row policies and penetration review
- Threat: CSRF/privilege escalation; Current boundary: CSRF and separate business/management scopes; Additional acceptance: Every new route and administrative flow
- Threat: Forged provider events; Current boundary: Platform authentication then active-call capability; Additional acceptance: Real envelopes/audio, replay and quota tests
- Threat: Prompt injection; Current boundary: Retrieved text stays data; permission stays in code; Additional acceptance: Adversarial cases and handoff evaluation
- Threat: Replayed side effects; Current boundary: Locks, idempotency, revisions and receipts; Additional acceptance: Cross-node contention and failure injection
- Threat: Unknown delivery; Current boundary: Explicit unknown state and receipt reconciliation; Additional acceptance: Non-calendar recovery expansion
- Threat: Documents/resource exhaustion; Current boundary: Isolated PDF parser and body/page/time/memory/concurrency limits; Additional acceptance: Global admission and multi-process tests
- Threat: Secret/supply-chain exposure; Current boundary: Lockfiles, private ignores, configuration tests and scanners; Additional acceptance: Advisory monitoring and rotation drills
- Threat: Host compromise; Current boundary: Non-root runtime, isolated services and readonly profiles; Additional acceptance: Central secrets, patching, least privilege and off-device backup

Secrets and telemetry

Use ignored environment files or an approved secret store. Secrets never belong in frontend settings, URLs, arguments, public logs or artifacts. Separate environments, providers and scopes. Rotate affected values and revoke compromised sessions.

Log bounded error classes, operation/revision/outcome and correlation IDs. Redact contacts, raw provider payloads, credentials and recordings. Avoid unbounded tenant/ call labels in metrics. Apply explicit audit access and retention.

Supply chain and release

Pin dependencies/actions, verify registries and review locks. CI uses minimal permissions, isolated test data, no production keys and no automatic deployment. Keep hooks and push protection active; resolve findings without bypasses. Deployment starts with drift detection, local changes/gates and a previous snapshot, then compatible rollout/readiness and hash parity. Exercise post-activation rollback and the actual served CSP. See reporting and incident runbooks.

Capacity roadmap: 1M+ concurrent users

This future target has staged capacity acceptance. The distributed architecture must be implemented and measured before increasing admission beyond a validated stage.

- Workload: Registered accounts; Definition: Stored identities and records; Measurements: Data/index volume and restore time
- Workload: Concurrent dashboard users; Definition: Active sessions and their read/write cadence; Measurements: Requests/sec, connections, p95/p99 and saturation
- Workload: Concurrent voice calls; Definition: Native audio sessions using business tools; Measurements: Concurrency/start/tool quotas and acoustic performance
- Workload: Mixed workload; Definition: Declared session/call split and tenant distribution; Measurements: Shared bottlenecks, fairness and delivery lag

One million sessions refreshing every thirty seconds imply roughly 33,333 requests/ second before writes. One million five-minute calls imply roughly 3,333 admissions/ second in steady state. These are sizing assumptions, excluding bursts/reconnects/ retries. Use concurrency = arrival rate × average lifetime with measured inputs.

Target topology

- Users and voice clients → CDN WAF admission
- CDN WAF admission → Regional routing
- Regional routing → Regional API cell A
- Regional routing → Regional API cell B
- Regional API cell A → Tenant-partitioned transactional storage
- Regional API cell B → Tenant-partitioned transactional storage

- Tenant-partitioned transactional storage → Partitioned durable workers
- Tenant-partitioned transactional storage → Partitioned durable workers
- Partitioned durable workers → Provider quota and backpressure controls
- Partitioned durable workers → Provider quota and backpressure controls

Separate failure domains and regional cells with explicit tenant placement. Keep one consistency authority/writer per booking resource. Session routing, regional failover and residency must agree. Add connection pooling, indexes, pagination, bounded caches and read replicas only with defined consistency. Narrow tenant locks only after concurrency regressions. A cache cannot confirm an uncommitted booking or invent availability.

Workers need partitioning, fairness, backpressure, receipt reconciliation and dead-letter handling. An external queue requires a transactional outbox and idempotent consumption. Admission controls provider concurrency/start/tool quotas, circuit breakers and bounded retry. Business HTTP capacity does not establish audio or telephony capacity.

Kubernetes is optional orchestration, requiring resource requests, readiness, disruption policies and measured scaling signals. Database/provider bottlenecks remain outside pod autoscaling. See the official autoscaling guide.

Stages

- Stage: C0; Intended scope: Guarded local baseline; Exit evidence: Environment/data, latency/errors, recovery and stop limits
- Stage: C1; Intended scope: 100–1,000 concurrent sessions; Exit evidence: Pagination, realistic cadence, pool/worker saturation
- Stage: C2; Intended scope: 10,000 concurrent sessions; Exit evidence: Multiple replicas, fairness, spike/soak and one-node loss
- Stage: C3; Intended scope: 100,000 concurrent sessions; Exit evidence: Partitions, database failover, quotas and headroom
- Stage: C4; Intended scope: 1M+ concurrent users; Exit evidence: Distributed generators, declared mixed workload, regional failure and sustained SLOs
- Stage: V1–V4; Intended scope: Separate increasing voice concurrency; Exit evidence: Written provider quotas, small real audio acceptance, staged call tests

These are planned gates, not hardware promises. Determine throughput/data size before instance counts. Reserve measured headroom for a failed replica/cell and account for tenant skew and retry amplification.

Safe tests and deliverables

Most tests use local controlled transports/isolated databases. Ramp gradually; cap duration, concurrency and data growth; stop at configured CPU/memory/pool/error/ latency limits. Measure generator saturation separately. Heavy tests must not slow shared live services or affect other projects.

Run baseline/ramp/spike/soak/recovery/failover against exact releases/configuration. Check duplicates and normal-user latency during load. Live voice requires approved costs/quotas and the smallest useful sample. High native traffic cannot be assumed free or inferred from HTTP tests.

Each stage delivers workload/data profiles, latency/resource charts, provider quota evidence, cost envelope, saturation diagnosis, failure drills and an admission ceiling. See load criteria, reliability and build-plan gates.

Reliability, uptime objectives and recovery

Future baseline: 99%. Production objective: 99.9% for declared service indicators. The build plan retains its stronger production gate. An SLA requires a separate contract, eligibility and remedies. Recorded observation scope is in deployment evidence.

- Indicator: API; Good event: Eligible request meets its result/latency threshold;
Measurement: Valid authorized requests; client errors recorded separately
- Indicator: Booking; Good event: Valid intent reaches calendar receipt before deadline;
Measurement: Accepted eligible intents; business conflict is distinct
- Indicator: Worker; Good event: Eligible job reaches receipt before deadline;
Measurement: Admitted jobs; unknown outcome is not success
- Indicator: Voice; Good event: Admitted call meets setup/audio/tool requirements;
Measurement: Calls within approved quota; setup/audio/tools separated
- Indicator: Website; Good event: Required routes remain usable within budget;
Measurement: Actual requests plus declared external probe cadence

Use rolling thirty-day and shorter burn-rate windows. Record workload/release, sample count, errors, latency histograms and unknown telemetry intervals. Keep time-based reachability separate from event-based availability. Health endpoints prove only their declared scope, not usable voice/external delivery. Missing samples are never silently successful.

In a thirty-day time-based window, 99% permits 432 minutes, 99.5% permits 216 minutes, and 99.9% permits 43.2 minutes of unavailability. Event budgets instead use eligible events $\times (1 - \text{objective})$. Measurement policy follows Google's SLO guidance.

Error budgets and redundancy

Define eligibility, thresholds, exclusions and queries before observing. Version them as operational configuration. Alert on fast/slow burn; exhausted budgets prioritize repair over risky releases. Exclusions remain visible; maintenance counts unless the declared contract explicitly says otherwise.

The current server is a host failure domain. Future releases add redundant API, independent workers, database failover and regional recovery with booking uniqueness, tenant and receipt-integrity drills. Provider failure needs bounded admission and human handoff; mid-call provider switching has its own native-flow contract.

Use readiness-driven rolling/canary release, compatible schemas, graceful draining and lease reconciliation. Expand/migrate/contract schema changes preserve rollback where possible. Test application/configuration/data compatibility. Zero-downtime rollout is an exercised goal, not a guarantee against every failure.

Disaster recovery

Planning objectives: RPO at most fifteen minutes and RTO at most sixty minutes for the defined failure class. Validate before contractual use. Evaluate WAL/PITR, encryption, independent storage, key recovery and regional placement; logical snapshots alone do not meet every recovery objective.

Restore data/configuration/migration state into an isolated target, compare counts/ hashes and exercise business flows. Record achieved RPO/RTO, duration and cleanup. An earlier backup's restore does not validate a later backup. Test unavailable primary storage and expired credentials. Agent checkpoints are recovery notes, not off-device backups or physical power-loss proof.

Monitoring and drills

Measure latency/errors, locks/pools, worker lag/leases/unknown outcomes, provider quotas, session errors and resources. Redact private data. Assign primary/backup incident owners and alerts. Drill node loss, worker crash after side effect, database failover, bad release, provider denial, telemetry loss and restore.

Operational runbooks

Use approved targets and credentials. Inspect actual state before replaying an interrupted action; a timestamp does not prove that an action failed.

API or deployment failure

Identify active release/image, gateway and dependencies. Inspect latency/errors, database pools/locks and resource signals. Compare live/local artifacts and save a before snapshot. Restore a pinned compatible release when needed; verify readiness, business flows and hash parity. Record interval, observation gaps and root cause.

Backlog or unknown delivery

Inspect revision, immutable intent, lease, attempts and receipt. Preserve user intent. Replay only known failed/dead work according to contract. Reconcile stable calendar IDs. Ambiguous CRM/email remains unknown until receipt evidence supports recovery. Never resend a confirmation merely because a process restarted.

Provider denial/quota

Stop identical rejected requests and retain sanitized status/type/correlation. Verify documented authentication, application headers and approved quotas/account state. Follow official support guidance. Do not evade security, enable billing or replace resources to force access. Apply bounded admission and handoff.

Security/privacy incident

Contain routes/credentials, revoke sessions and rotate compromised values. Assess tenant/data/provider scope and preserve private audit evidence. Involve assigned security/privacy owners and assess actual notification duties/deadlines. Never publish participant data or secrets. Add corrective regression tests.

Restore/restart

Verify backup integrity/encryption/keys; restore separately and compare records, migrations and business flows. Measure RPO/RTO before traffic switch. Inspect pending/unknown work before worker start. Never delete working volumes or prune shared infrastructure as recovery.

Records

Record intended action, baseline, recovery, commands/action IDs, outcome, unresolved state and next step. Private evidence stays private. Public results derive from verified evidence. Google Docs keep stable identity; canonical PDFs change only after a valid temporary export is ready.

Testing and verification

Frontend gates: tests, TypeScript, ESLint, formatting, Vite build, exact browser artifact and release/contract checks. Backend: pytest, strict Mypy, Ruff, Bandit, Gitleaks, Semgrep and wheel/sdist. Advisory checks run before releases/lock changes.

Install and verify commit hooks

Install pre-commit and provide gitleaks, bandit and Python on the commit process's PATH. Semgrep must be installed on Linux; Windows commits use the installed WSL Linux scanner through the public platform adapter. Windows also needs wsl.exe, wslpath and a working Bash login environment that resolves Semgrep. Use the versions exercised in publication evidence and verify their official package/release sources when upgrading.

```
python -m pre_commit install
```

```
python -m pre_commit run --all-files
```

Verify that all three named scanner hooks actually run and pass. Do not bypass the hook with disabled hooks or --no-verify. The public configuration uses system tools, so installing

pre-commit alone does not install each scanner. CI separately installs pinned scanners; it is a backstop rather than a substitute for local hooks.

The backend suite loads `.local/core-engine/.env`. Configure an isolated loopback PostgreSQL administrator allowed to create/drop generated `voicebridge_test_` databases; never point it at production. Tests replace service credentials and reject external sockets. Process-crash follow-up tests also require a local capture SMTP service on the host/port configured in that test environment; it must not relay to real recipients. CI creates isolated PostgreSQL and digest-pinned Mailpit services and safe settings from `.env.example`; real provider execution remains disabled.

Exercise actual HTTP/CLI/browser sign-in/CSRF/logout, booking retries/conflicts, knowledge sources, worker lease/crash recovery, tenant/scope denial, provider authentication and terminal retirement. Check the exact artifact and served CSP. Restore separately and compare hashes/counts plus business behavior.

Native acceptance adds setup, intent/bind, audio, interruption, grounded answers, tools, handoff, transcripts and end-call durability. Keep approved live samples minimal; ordinary gates require no paid provider traffic.

Capacity uses guarded baseline/ramp/soak/failure scenarios within approved resource limits. Publication checks worktree, staging, full history, private exclusions, metadata and attribution; fresh-context review records scope and verdict. Public CI uses standard Linux runners, minimal permissions and no production keys, private uploads or automatic deployment. Counts remain dated to their evidence.

Delivery roadmap

The build plan contains fifty implementation and thirty readiness criteria. Existing local/deployed stages have separate focused acceptance records.

- Workstream: Business engine; Foundation: Persistent bookings, knowledge, jobs and tools; Next acceptance: Production receipts and ambiguous-delivery recovery
- Workstream: Frontend; Foundation: Public/operator pages and HTTP console; Next acceptance: Route splitting, paging and more persistent operator flows
- Workstream: Native voice; Foundation: Retell/Vapi profiles/auth/lifecycle; Next acceptance: Intent/native bind/audio/tool/end-call flows on both platforms
- Workstream: Custom voice; Foundation: ElevenLabs settings boundary; Next acceptance: Rights, quality, pronunciation, latency and compatibility
- Workstream: Identity; Foundation: Owner OIDC and revocable sessions; Next acceptance: Customer onboarding, MFA/recovery and role administration
- Workstream: Legal/privacy; Foundation: Applicability register; Next acceptance: Customer decisions, retention/deletion and contracts
- Workstream: Capacity; Foundation: Guarded measurements; Next acceptance: 1K -> 10K -> 100K -> 1M+ sessions; separate voice/mixed stages
- Workstream: Reliability; Foundation: Immutable releases, parity, observer and restore; Next acceptance: Redundancy, full SLO window, off-device backup and RPO/RTO

- Workstream: Security; Foundation: Config/secret checks, scanners and access tests; Next acceptance: Penetration review, distributed limits and rotation drills
- Workstream: Delivery; Foundation: Public source/docs and free CI; Next acceptance: Versioned evidence, compatibility and reproducible handoff

Order: native/connector correctness -> customer identity/data lifecycle -> pagination/limits/telemetry -> replicas -> database/worker failover -> staged capacity. High voice concurrency requires provider/telephony capacity agreements. Dates and instance counts cannot substitute for exit evidence. Each milestone records release/workload/tests/review/dependencies. Chargeable tests need approval.

Third-party materials and rights

Original project material is copyright Zahidul Islam; LICENSE preserves the author's rights. An open-source licence has not been selected.

DM Sans and Manrope retain SIL Open Font License notices in apps/web/public/fonts/DM-Sans-OFL.txt and Manrope-OFL.txt. Keep them with font distribution. Dependency packages retain their bundled licences; npm/Python locks identify selected versions. Review notices when distributing a product.

Provider names describe integration targets, not endorsement. Voices/clones/recordings need separate rights/contracts. Source rights do not authorize voice assets or client material. Visual media needs provenance/permitted-use review before redistribution; publication makes no third-party clearance claim. See legal/privacy.